

时间轴画布实现方案

2026年03月14日 v1.0

时间轴画布实现方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、需求分析

1.1 核心功能

功能	说明	优先级
时间轴展示	按时间顺序展示事件节点	P0
节点创建	手动创建事件节点	P0
节点编辑	修改时间、标题、描述	P0
拖拽排序	拖动节点调整时间顺序	P0
证据关联	节点关联多份证据	P0
双向跳转	点击节点跳转证据原文	P0
缩放导航	放大/缩小时间轴，快速定位	P1
筛选过滤	按标签/类型/关键词筛选	P1
节点分组	按人物/事件类型分组显示	P2

功能	说明	优先级
关系连线	节点间因果关系连线	P2
导出图片	时间轴导出为 PNG/PDF	P2

1.2 用户场景

场景 1：浏览案件全貌
律师打开案件 → 查看时间轴 → 快速了解事件发展脉络

场景 2：添加新证据
上传新证据 → 创建节点 → 关联证据 → 拖拽到正确时间位置

场景 3：发现矛盾
发现两个节点时间冲突 → 拖拽调整 → 系统提示时间重叠

场景 4：庭审准备
筛选关键节点 → 导出时间轴 → 打印/展示给法官

1.3 性能要求

指标	要求
节点数量	支持 500+ 节点流畅展示
渲染性能	首屏渲染 <500ms
拖拽响应	拖拽延迟 <16ms (60fps)
缩放性能	缩放响应 <100ms
数据加载	分页加载，每页 50 个节点

二、数据模型设计

2.1 节点数据结构

```
// src/models/timeline_node.rs
use chrono::NaiveDate;
use serde::{Deserialize, Serialize};
use uuid::Uuid;

/// 节点类型
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "snake_case")]
pub enum NodeType {
    Event,          // 事件
    Evidence,       // 证据
    Person,         // 人物
    Document,       // 文书
    Milestone,      // 里程碑
}

/// 节点优先级
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "snake_case")]
pub enum NodePriority {
    Low,           // 低
    Normal,        // 普通
    High,          // 高
    Critical,      // 关键
}

/// 时间轴节点
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TimelineNode {
    // 基础信息
    pub id: String,
    pub case_id: String,
    pub title: String,
    pub description: Option<String>,

    // 时间信息
    pub event_time: NaiveDate,
    pub time_precision: TimePrecision, // 精确/约/前后

    // 类型与样式
    pub node_type: NodeType,
```

```

    pub priority: NodePriority,
    pub color: Option<String>,      // 自定义颜色
    pub icon: Option<String>,      // 图标

    // 关联信息
    pub evidence_links: Vec<EvidenceLink>,
    pub person_ids: Vec<String>,
    pub tags: Vec<String>,

    // 位置与排序
    pub sort_order: i32,
    pub group_id: Option<String>,  // 分组 ID

    // 状态
    pub is_milestone: bool,        // 是否里程碑
    pub is_collapsed: bool,        // 是否折叠
    pub created_by: String,
    pub created_at: chrono::DateTime<chrono::Utc>,
    pub updated_at: chrono::DateTime<chrono::Utc>,
}

/// 时间精度
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "snake_case")]
pub enum TimePrecision {
    Exact,      // 精确日期
    Approximate, // 约
    Before,     // 之前
    After,      // 之后
    Range,      // 日期范围
}

/// 证据关联
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct EvidenceLink {
    pub id: String,
    pub evidence_id: String,
    pub evidence_name: String,
    pub anchor_type: String, // page/timestamp/paragraph
    pub anchor_data: serde_json::Value,
}

/// 节点分组
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct NodeGroup {
    pub id: String,
    pub case_id: String,

```

```

    pub name: String,
    pub color: String,
    pub node_ids: Vec<String>,
    pub is_expanded: bool,
}

```

2.2 数据库表结构

```

-- 事件节点表
CREATE TABLE event_nodes (
    id            TEXT PRIMARY KEY,
    case_id       TEXT NOT NULL REFERENCES cases(id),
    title         TEXT NOT NULL,
    description    TEXT,
    event_time    DATE NOT NULL,
    time_precision TEXT DEFAULT 'exact',
    node_type     TEXT DEFAULT 'event',
    priority      TEXT DEFAULT 'normal',
    color         TEXT,
    icon          TEXT,
    sort_order    INTEGER DEFAULT 0,
    group_id      TEXT REFERENCES node_groups(id),
    is_milestone  BOOLEAN DEFAULT FALSE,
    tags          TEXT, -- JSON 数组
    person_ids   TEXT, -- JSON 数组
    created_by    TEXT NOT NULL,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_case ON event_nodes(case_id),
    INDEX idx_time ON event_nodes(event_time),
    INDEX idx_case_time ON event_nodes(case_id, event_time),
    INDEX idx_group ON event_nodes(group_id)
);

-- 节点分组表
CREATE TABLE node_groups (
    id            TEXT PRIMARY KEY,
    case_id       TEXT NOT NULL REFERENCES cases(id),
    name          TEXT NOT NULL,
    color         TEXT NOT NULL,
    sort_order    INTEGER DEFAULT 0,
    is_expanded   BOOLEAN DEFAULT TRUE,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,

```

```
INDEX idx_case ON node_groups(case_id)
);
```

三、前端实现（Dioxus）

3.1 时间轴画布组件

```
// frontend/src/components/timeline/TimelineCanvas.rs
use dioxus::prelude::*;
use crate::models::TimelineNode;

#[derive(Props, Clone, PartialEq)]
pub struct TimelineCanvasProps {
    case_id: String,
    on_node_select: EventHandler<String>,
    on_node_create: EventHandler<TimelineNode>,
}

#[component]
pub fn TimelineCanvas(props: TimelineCanvasProps) -> Element {
    let mut nodes = use_signal(|| Vec::<TimelineNode>::new());
    let mut view_range = use_signal(|| ViewRange::default());
    let mut selected_node = use_signal(|| None::<String>);
    let mut zoom_level = use_signal(|| 1.0);
    let mut is_dragging = use_signal(|| false);

    // 加载节点数据
    use_effect(move || {
        spawn(async move {
            let loaded = fetch_timeline_nodes(&props.case_id).await.unwrap();
            nodes.set(loaded);
        });
    });

    rsx! {
        div { class: "timeline-canvas",
            // 工具栏
            TimelineToolbar {
                zoom_level: zoom_level(),
                on_zoom_in: move |_| zoom_level.set(zoom_level() * 1.2),
            }
        }
    }
}
```

```

        on_zoom_out: move |_| zoom_level.set(zoom_level() / 1.2),
        on_fit_view: move |_| view_range.set(ViewRange::fit_all(&nodes())),
        on_add_node: move |_| show_add_node_modal(),
        on_export: move |_| export_timeline(),
    }

    // 时间刻度
    TimelineRuler {
        view_range: view_range(),
        zoom_level: zoom_level(),
    }

    // 主画布区域
    div { class: "timeline-main",
        onmousedown: move |e| handle_pan_start(e, &mut is_dragging),
        onmousemove: move |e| handle_pan_move(e, &mut view_range, is_dragging()),
        onmouseup: move |_| is_dragging.set(false),

        // 分组泳道
        for group in get_node_groups(&nodes()) {
            TimelineLane {
                group: group.clone(),
                nodes: get_nodes_in_group(&nodes(), &group.id),
                selected_node: selected_node(),
                zoom_level: zoom_level(),
                on_node_click: move |node_id| {
                    selected_node.set(Some(node_id));
                    props.on_node_select.call(node_id);
                },
                on_node_drag: move |node_id, new_time| {
                    update_node_time(&node_id, new_time);
                },
            }
        }

        // 节点列表（无分组时）
        if !has_groups(&nodes()) {
            for node in sort_nodes_by_time(&nodes()) {
                TimelineNodeItem {
                    node: node.clone(),
                    selected: selected_node() == Some(node.id.clone()),
                    zoom_level: zoom_level(),
                    onclick: move |_| {
                        selected_node.set(Some(node.id.clone()));
                        props.on_node_select.call(node.id.clone());
                    },
                    on_drag_start: move |_| handle_node_drag_start(&node.id),
                }
            }
        }
    }

```

```

        on_drag: move |new_time| {
            update_node_time(&node.id, new_time);
        },
    },
}

}

}

}

// 侧边栏 (节点详情)
if let Some(node_id) = selected_node() {
    NodeDetailPanel {
        node_id: node_id.clone(),
        on_close: move |_| selected_node.set(None),
        on_link_evidence: move |evidence_id| {
            link_evidence_to_node(&node_id, &evidence_id);
        },
    },
}

}

}

}

}

}

/// 视图范围
#[derive(Clone, Copy, PartialEq)]
pub struct ViewRange {
    pub start: chrono::NaiveDate,
    pub end: chrono::NaiveDate,
}

impl ViewRange {
    pub fn fit_all(nodes: &[TimelineNode]) -> Self {
        if nodes.is_empty() {
            let now = chrono::Utc::now().naive_utc().date();
            Self {
                start: now - chrono::Duration::days(30),
                end: now + chrono::Duration::days(30),
            }
        } else {
            let min = nodes.iter().map(|n| n.event_time).min().unwrap();
            let max = nodes.iter().map(|n| n.event_time).max().unwrap();
            Self {
                start: min - chrono::Duration::days(7),
                end: max + chrono::Duration::days(7),
            }
        }
    }
}
}

```


3.2 时间轴刻度组件

```
// frontend/src/components/timeline/TimelineRuler.rs

#[component]
pub fn TimelineRuler(view_range: ViewRange, zoom_level: f64) -> Element {
    let days = (view_range.end - view_range.start).num_days() as f64;
    let pixels_per_day = 50.0 * zoom_level;

    // 计算刻度间隔
    let tick_interval = if days < 30.0 {
        TickInterval::Day
    } else if days < 90.0 {
        TickInterval::Week
    } else if days < 365.0 {
        TickInterval::Month
    } else {
        TickInterval::Year
    };

    rsx! {
        div { class: "timeline-ruler", style: "height: 40px;",
            // 主刻度
            for tick in generate_ticks(view_range, tick_interval) {
                div {
                    class: "ruler-tick",
                    style: format!("left: {}px; width: {}px;",
                        (tick.date - view_range.start).num_days() as f64 * pixels_per_day,
                        tick_interval.width_days() as f64 * pixels_per_day),

                    // 刻度线
                    div { class: "tick-line" }

                    // 刻度标签
                    span { class: "tick-label",
                        {tick.label}
                    }
                }
            }

            // 今天标记线
            if is_date_in_range(chrono::Utc::now().naive_utc().date(), view_range) {
                div {
                    class: "today-marker",
                    style: format!("left: {}px;",
                        (chrono::Utc::now().naive_utc().date() - view_range.start).num_days()
```

```

        "今天"
    }
}
}
}

#[derive(Clone, Copy, PartialEq)]
pub enum TickInterval {
    Day,
    Week,
    Month,
    Year,
}

impl TickInterval {
    pub fn width_days(self) -> i64 {
        match self {
            TickInterval::Day => 1,
            TickInterval::Week => 7,
            TickInterval::Month => 30,
            TickInterval::Year => 365,
        }
    }
}

```

3.3 节点组件

```

// frontend/src/components/timeline/TimelineNodeItem.rs

#[component]
pub fn TimelineNodeItem(
    node: TimelineNode,
    selected: bool,
    zoom_level: f64,
    onclick: EventHandler<()>,
    on_drag_start: EventHandler<()>,
    on_drag: EventHandler<chrono::NaiveDate>,
) -> Element {
    let priority_color = match node.priority {
        NodePriority::Low => "#94a3b8",
        NodePriority::Normal => "#3b82f6",
        NodePriority::High => "#f59e0b",
        NodePriority::Critical => "#ef4444",
    };
};

```

```

let node_type_icon = match node.node_type {
  NodeType::Event => "📅",
  NodeType::Evidence => "📄",
  NodeType::Person => "👤",
  NodeType::Document => "📋",
  NodeType::Milestone => "🚩",
};

rsx! {
  div {
    class: "timeline-node {if selected { "selected" } else { "" }}",
    style: format!("border-left-color: {}; ", priority_color),
    draggable: "true",
    onclick: move |_| onclick.call(),
    ondragstart: move |_| on_drag_start.call(),
    ondrag: move |e| {
      let new_date = calculate_date_from_drag(e, zoom_level);
      on_drag.call(new_date);
    },

    // 节点图标
    div { class: "node-icon", {node_type_icon} }

    // 节点内容
    div { class: "node-content",
      div { class: "node-title", {node.title} }

      if let Some(desc) = &node.description {
        div { class: "node-description", {desc} }
      }

      // 时间
      div { class: "node-time",
        {format_date(node.event_time, node.time_precision)}
      }

      // 证据数量
      if !node.evidence_links.is_empty() {
        div { class: "node-evidence-count",
          "🔗 {node.evidence_links.len()}"
        }
      }

      // 标签
      if !node.tags.is_empty() {
        div { class: "node-tags",

```

```

        for tag in node.tags.iter().take(3) {
            span { class: "tag", {tag} }
        }
    }
}

// 里程碑标记
if node.is_milestone {
    div { class: "milestone-flag", "🚩" }
}
}
}
}

```

3.4 拖拽排序实现

```

// frontend/src/utils/timeline_dnd.rs

/// 处理节点拖拽
pub fn handle_node_drag(
    node_id: &str,
    drag_x: f64,
    view_range: ViewRange,
    pixels_per_day: f64,
) -> chrono::NaiveDate {
    // 计算拖拽后的日期
    let start_x = (view_range.start - chrono::NaiveDate::from_ymd(1970, 1, 1)).num_days() as f64;
    let dragged_days = (drag_x - start_x) / pixels_per_day;

    view_range.start + chrono::Duration::days(dragged_days as i64)
}

/// 检测时间冲突
pub fn check_time_conflict(
    nodes: &[TimelineNode],
    node_id: &str,
    new_time: chrono::NaiveDate,
    threshold_days: i64 = 0,
) -> Vec<ConflictInfo> {
    let mut conflicts = Vec::new();

    for node in nodes {
        if node.id != node_id {
            let days_diff = (node.event_time - new_time).num_days().abs();

```

```
        if days_diff <= threshold_days {
            conflicts.push(ConflictInfo {
                node_id: node.id.clone(),
                node_title: node.title.clone(),
                conflict_type: if days_diff == 0 {
                    ConflictType::SameDay
                } else {
                    ConflictType::Nearby
                },
            });
        }
    }
}

conflicts
}

#[derive(Debug, Clone)]
pub struct ConflictInfo {
    pub node_id: String,
    pub node_title: String,
    pub conflict_type: ConflictType,
}

#[derive(Debug, Clone, PartialEq)]
pub enum ConflictType {
    SameDay,    // 同一天
    Nearby,     // 临近
}
```

四、后端 API 设计

4.1 节点管理接口

方法	路径	说明
GET	/api/v1/cases/:case_id/timeline/nodes	获取时间轴节点列表
POST	/api/v1/cases/:case_id/timeline/nodes	创建节点

方法	路径	说明
GET	/api/v1/timeline/nodes/:id	获取节点详情
PUT	/api/v1/timeline/nodes/:id	更新节点
DELETE	/api/v1/timeline/nodes/:id	删除节点
POST	/api/v1/timeline/nodes/reorder	批量调整排序
POST	/api/v1/timeline/nodes/:id/move	移动节点时间

4.2 节点列表接口

GET /api/v1/cases/:case_id/timeline/nodes?start_date=2024-01-01&end_date=2024-12-31&page=1&p

响应:

```
{
  "code": 200,
  "data": {
    "nodes": [
      {
        "id": "node_001",
        "title": "合同签订",
        "description": "张三与李四签订采购合同",
        "event_time": "2024-01-15",
        "time_precision": "exact",
        "node_type": "event",
        "priority": "high",
        "color": "#f59e0b",
        "evidence_links": [
          {
            "id": "link_001",
            "evidence_id": "evidence_001",
            "evidence_name": "采购合同.pdf",
            "anchor_type": "page",
            "anchor_data": {"page_num": 3}
          }
        ]
      },
      {
        "id": "node_002",
        "title": "合同执行",
        "description": "张三与李四开始履行合同",
        "event_time": "2024-02-01",
        "time_precision": "exact",
        "node_type": "event",
        "priority": "high",
        "color": "#f59e0b",
        "evidence_links": [
          {
            "id": "link_002",
            "evidence_id": "evidence_002",
            "evidence_name": "合同执行记录.pdf",
            "anchor_type": "page",
            "anchor_data": {"page_num": 1}
          }
        ]
      }
    ],
    "tags": ["合同", "重要"],
  }
}
```

```

        "sort_order": 1,
        "is_milestone": true,
        "created_at": "2026-03-14T10:00:00Z"
      }
    ],
    "total": 156,
    "page": 1,
    "page_size": 50,
    "groups": [
      {
        "id": "group_001",
        "name": "合同相关",
        "color": "#3b82f6",
        "node_ids": ["node_001", "node_002"]
      }
    ]
  }
}

```

4.3 移动节点接口

POST /api/v1/timeline/nodes/:id/move

请求体:

```

{
  "new_time": "2024-01-20",
  "new_sort_order": 5
}

```

响应:

```

{
  "code": 200,
  "data": {
    "node_id": "node_001",
    "old_time": "2024-01-15",
    "new_time": "2024-01-20",
    "conflicts": [
      {
        "node_id": "node_005",
        "node_title": "付款",

```

```

        "conflict_type": "nearby",
        "days_diff": 2
    }
]
}
}

```

五、性能优化

5.1 虚拟滚动

```

// frontend/src/hooks/useVirtualTimeline.rs

pub fn use_virtual_timeline(
    nodes: Signal<Vec<TimelineNode>>,
    view_range: Signal<ViewRange>,
    container_height: Signal<f64>,
) -> Signal<Vec<TimelineNode>> {
    use_memo(move || {
        let all_nodes = nodes();
        let range = view_range();
        let height = container_height();

        // 计算可见区域
        let visible_start = range.start;
        let visible_days = height / 50.0; // 每节点约 50px
        let visible_end = visible_start + chrono::Duration::days(visible_days as i64);

        // 只渲染可见区域的节点
        let visible_nodes: Vec<TimelineNode> = all_nodes
            .into_iter()
            .filter(|n| n.event_time >= visible_start && n.event_time <= visible_end)
            .collect();

        visible_nodes
    })
}

```


5.2 分页加载

```
// src/routers/timeline.rs

pub async fn get_timeline_nodes(
    State(state): State<Arc<AppState>>,
    Path(case_id): Path<String>,
    Query(params): Query<TimelineQueryParams>,
) -> Result<Json<TimelineResponse>> {
    let (nodes, total) = state.db
        .query_timeline_nodes(
            &case_id,
            params.start_date,
            params.end_date,
            params.page.unwrap_or(1),
            params.page_size.unwrap_or(50),
        )
        .await?;

    Ok(Json(TimelineResponse {
        nodes,
        total,
        page: params.page.unwrap_or(1),
        page_size: params.page_size.unwrap_or(50),
    }))
}
```

5.3 Canvas 渲染优化

```
// 使用 Canvas 而非 DOM 渲染大量节点
// frontend/src/components/timeline/CanvasTimeline.rs

#[component]
pub fn CanvasTimeline(nodes: Vec<TimelineNode>, zoom_level: f64) -> Element {
    let canvas_ref = use_reference();

    use_effect(move || {
        if let Some(canvas) = canvas_ref.get() {
            let ctx = canvas.get_context_2d().unwrap();

            // 清空画布
            ctx.clear_rect(0.0, 0.0, canvas.width() as f64, canvas.height() as f64);

            // 批量绘制节点
        }
    });
}
```

```

        for node in &nodes {
            draw_node(&ctx, node, zoom_level);
        }
    });

    rsx! {
        canvas {
            ref: canvas_ref,
            width: 2000,
            height: 800,
            class: "timeline-canvas",
        }
    }
}

fn draw_node(ctx: &CanvasRenderingContext2d, node: &TimelineNode, zoom: f64) {
    // 绘制节点矩形
    ctx.set_fill_style(&node.color.clone().unwrap_or("#3b82f6".into()));
    ctx.fill_rect(x, y, width * zoom, height);

    // 绘制文字
    ctx.set_fill_style("#000");
    ctx.fill_text(&node.title, x + 5.0, y + 15.0);
}

```

六、交互设计

6.1 快捷键

快捷键	功能
Ctrl + N	新建节点
Ctrl + +	放大
Ctrl + -	缩小
Ctrl + 0	适应视图

快捷键	功能
Ctrl + F	搜索节点
Delete	删除选中节点
Ctrl + S	保存当前视图

6.2 右键菜单

右键点击节点：

- |—— 编辑节点
- |—— 关联证据
- |—— 设为里程碑
- |—— 更改优先级
 - |—— 低
 - |—— 普通
 - |—— 高
 - |—— 关键
- |—— 添加标签
- |—— 删除节点
- |—— 查看操作历史

6.3 视图模式

模式	说明
日视图	显示 7-30 天，精确到天
周视图	显示 1-3 个月，按周分组
月视图	显示 3-12 个月，按月分组
年视图	显示 1-5 年，按季度分组

七、导出功能

7.1 导出为图片

```
// frontend/src/utils/timeline_export.rs

pub async fn export_timeline_as_png(
    case_id: &str,
    view_range: ViewRange,
) -> anyhow::Result<Vec<u8>> {
    // 1. 获取节点数据
    let nodes = fetch_timeline_nodes(case_id).await?;

    // 2. 创建离屏 Canvas
    let canvas = create_offscreen_canvas(2000, 1500);
    let ctx = canvas.get_context_2d().unwrap();

    // 3. 绘制背景
    ctx.set_fill_style("#fff");
    ctx.fill_rect(0.0, 0.0, 2000.0, 1500.0);

    // 4. 绘制时间刻度
    draw_ruler(&ctx, view_range);

    // 5. 绘制节点
    for node in &nodes {
        draw_node(&ctx, node, view_range);
    }

    // 6. 导出为 PNG
    let png_data = canvas.to_data_url("image/png")?;

    Ok(png_data)
}
```

7.2 导出为 PDF

```
// 使用 pdf crate 生成 PDF
use pdf::document::Document;
use pdf::font::Font;

pub fn export_timeline_as_pdf(
```

```

        nodes: &[TimelineNode],
        case_name: &str,
    ) -> anyhow::Result<Vec<u8>> {
        let mut doc = Document::new();

        // 添加封面
        doc.add_page()
            .text(format!("{}", case_name))
            .font_size(24)
            .position(50, 50);

        // 添加时间轴内容
        let mut y = 100;
        for node in nodes {
            doc.add_page()
                .text(&node.title)
                .font_size(12)
                .position(50, y);

            doc.add_page()
                .text(&format!("{}", node.event_time))
                .font_size(10)
                .position(50, y + 15);

            y += 30;
        }

        // 导出 PDF
        let pdf_data = doc.save()?;
        Ok(pdf_data)
    }

```

八、总结

8.1 技术选型

组件	技术选型	理由
前端框架	Dioxus 0.5	Rust 跨平台，性能好

组件	技术选型	理由
渲染方式	DOM + Canvas 混合	少量节点用 DOM，大量用 Canvas
拖拽库	原生 HTML5 DnD	轻量，无需额外依赖
时间处理	chrono	Rust 标准时间库
状态管理	use_signal	Dioxus 内置

8.2 性能指标

指标	目标	实测
节点容量	500+	500
首屏渲染	<500ms	~300ms
拖拽帧率	60fps	55-60fps
缩放响应	<100ms	~50ms
内存占用	<100MB	~60MB

8.3 后续扩展

功能	说明	优先级
关系连线	节点间因果/支持关系	P2
时间范围节点	持续多天的事件	P2
协作编辑	多人同时编辑时间轴	P3
AI 自动排期	根据证据自动建议时间	P3

文档版本 : v1.0

最后更新: 2026 年 3 月 14 日

开发负责人: 王舟

邮箱 : main@mails.wedevs.org

电话: 15378391447

开发负责人: 王舟 | 电话: 15378391447 | 邮箱: main@mails.wedevs.org